

WELD

Especificación Técnica e Arquitectura del Cliente HTTP End-to-End Definitivo

TYPESCRIPT NATIVO

ZERO-CONFIG

OFFLINE-FIRST

TYPE-SAFE

WELD es un cliente HTTP de última generación diseñado para el desarrollo web moderno. Su concepto central se basa en la idea de una "soldadura" rígida y confiable entre el servidor (Backend) y el cliente (Frontend), garantizando que los contratos de datos se respeten estrictamente tanto en tiempo de compilación como en tiempo de ejecución.

A diferencia de soluciones genéricas tradicionales, WELD implementa una filosofía de **Complejidad Progresiva**: es lo suficientemente simple como para ser usado en una landing page de bajo presupuesto con configuración cero, y lo suficientemente elástico y robusto como para actuar como el adaptador de infraestructura principal en un Monolito Modular con Arquitectura Hexagonal de escala empresarial.

1. Los 4 Pilares de Innovación (Estrategia de Nicho)

WELD unifica cuatro dolores de cabeza históricos del desarrollo frontend en un único pipeline de ejecución unificado de peso pluma:

- **End-to-End (E2E) Type Safety Definitivo (Tiempo de Compilación):** Mediante el uso de tipos mapeados avanzados, WELD puede absorber el `AppRouter` o contrato de rutas del backend. El desarrollador obtiene autocompletado exacto y validación estricta de rutas, parámetros de consulta (query params), cabeceras y cuerpos directamente en el IDE. Si el backend cambia, el código frontend falla antes de compilar. *Costo en el bundle final: 0 KB.*
- **Validación Estricta en Tiempo de Ejecución (Zod Nativo):** Los tipos de TypeScript desaparecen al compilar. WELD utiliza esquemas de Zod en la frontera de la red para validar que las respuestas del servidor coincidan con el contrato real en tiempo de ejecución, aislando a la interfaz de usuario de corrupciones de datos o fallos silenciosos de la API.
- **Resiliencia Offline-First con Cola de Sincronización:** Totalmente autónomo. Para peticiones idempotentes (GET), WELD sirve datos locales persistidos de forma segura a través de IndexedDB cuando no hay red. Para mutaciones de estado (POST, PUT, DELETE), retiene ordenadamente las acciones en una cola asíncrona local, disparándolas automáticamente en segundo plano cuando el evento `navigator.onLine` se restablece.
- **Deduplicación de Peticiones Concurrentes en Vuelo:** Optimiza el rendimiento de forma nativa. Si múltiples componentes en la interfaz de usuario solicitan el mismo recurso en el mismo instante de tiempo ($\Delta t \rightarrow 0$), WELD unifica las peticiones y las enlaza a una única promesa en vuelo, reduciendo drásticamente el tráfico innecesario en el servidor.

2. Flujo de Datos y Pipeline de Ejecución

El ciclo de vida de cualquier petición HTTP en WELD pasa de forma lineal por la siguiente estructura secuencial:



3. Flexibilidad Elástica: De Pequeño a Grande

WELD escala de manera fluida y orgánica según la envergadura del software:

Escala del Proyecto	Requisitos Típicos	Comportamiento de WELD
Pequeño (Landing Pages, MVP simples, Sitios con Vite)	Peticiones rápidas, código directo, sin boilerplate ni configuraciones complejas.	Zero-Config: Retorna JSON crudo y promesas directas. Las señales reactivas se manejan de fondo para los estados de carga básicos.
Mediano (SaaS tradicionales, SPAs con Firebase/ Supabase)	Garantía de tipos mínimos en UI, protección ante clicks dobles que saturen la base de datos.	Validación Integrada: Se introducen esquemas de Zod puntuales. La deduplicación en vuelo se activa por defecto para optimizar renderizados.
Grande / Enterprise (Monolitos Modulares, Arquitectura Hexagonal)	Sincronización robusta offline, desacoplamiento estricto de capas, seguridad E2E de punta a punta.	Enterprise Mode: Inyección total de tipos de backend, almacenamiento transaccional en IndexedDB y mappers de infraestructura nativos.

4. Adaptación en Arquitectura Hexagonal

Para entornos de alta ingeniería que utilizan **Monolitos Modulares** y patrones de **Arquitectura Hexagonal (Puertos y Adaptadores)**, WELD ofrece ventajas arquitectónicas insustituibles:

Ubicación Estricta en la Capa de Infraestructura

WELD vive exclusivamente en los límites exteriores del hexágono (Adaptadores de Salida o *Driven Adapters*). El dominio o núcleo de negocio de tu módulo jamás se entera de la existencia de WELD, manteniendo los casos de uso puros y desacoplados.

Los esquemas de Zod que recibe WELD actúan como los mappers de infraestructura ideales: interceptan el Objeto de Transferencia de Datos (DTO) que proviene del servidor, comprueban su validez en la frontera y permiten su transformación limpia hacia entidades puras de dominio. Toda la complejidad de reintentos, colas de mutaciones sin conexión e IndexedDB queda encapsulada sin ensuciar la lógica de negocio.

5. Compatibilidad Absoluta y Distribución

WELD se compila utilizando herramientas modernas de alto rendimiento (tsup / esbuild) para generar salidas híbridas simultáneas:

- **Compatibilidad de Ecosistema:** Soporte nativo para **ES Modules (.mjs)** y **CommonJS (.js)** con generación automática de mapas de definición de tipos (.d.ts). Funciona de forma transparente en JavaScript puro y TypeScript.
- **Frameworks y Librerías:** React (Next.js Edge y SSR), Vue 3 (Nuxt), Angular, Svelte y SolidJS.
- **Entornos de Ejecución Híbridos:** React Native, Expo, Capacitor (donde el pilar Offline-First es crítico), Electron, Node.js (18+), Bun y Deno.

6. Ejemplo Práctico de Implementación Definitiva

```
import { Weld } from 'weld-http';
import { z } from 'zod';

// 1. Contrato del Backend (Compartido o Inferred)
type AppBackendRouter = {
  'v1/products': {
    GET: { response: { id: string; name: string; price: number }[] }
    POST: { body: { name: string; price: number }; response: { id: string } }
  }
};

// 2. Esquema de Validación de Respuesta (Tiempo de ejecución)
const ProductSchema = z.object({
  id: z.string(),
  name: z.string(),
  price: z.number()
});

// 3. Inicialización del Cliente Elástico
const api = new Weld<AppBackendRouter>('https://api.tuempresa.com');

// 4. Consumo E2E Seguro, Reactivo y Resiliente Offline
const { promise, signal } = api.get('v1/products', z.array(ProductSchema), {
  offlineFallback: true
});
```